Modul Pelatihan BERT Dari Nol

https://share.gemini.google/QulxOtmRcNLJ

Dibuat dengan **4 Argon** 5 November 2026 pukul 21.12 • Dipublikasikan 5 November 2026 pukul 21.25

Bert-Kelompok1

Kamu adalah asisten pair-programming PyTorch untuk proyek akademik **BERT-from-Scratch (Kelompok 12)** menggunakan metode **Vibe Coding** (kolaborasi bertahap, transparan, mudah dipahami dan di-deb

Rancangan modul pelatihan (`training/`) untuk proyek **BERT-from-Scratch (Kelompok 12)** disusun dengan mengikuti penuh kontrak `SPEC.md` v1.0 tanpa menggunakan komponen instan `transformers` .

1. Alur Matematis, Dimensi Tensor, dan Penanganan Edge-Case

A. Alur Tensor Lintas Modul `(B, L, V)`

1. **Input Batch (`data/data.py`):**
   Setiap iterasi `DataLoader` menghasilkan dictionary berisi 4 tensor `torch.long` berdimensi `(B, L)` denga) dan $L = \text{max_seq_length}$ (`128`):

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

`attention_mask` : `(B, L)` — Bernilai `1` untuk token asli dan `0` untuk `[PAD]` .

`token_type_ids` : `(B, L)` — Bernilai `0` untuk segmen tunggal MLM.

`labels` : `(B, L)` — Berisi ID token asli pada posisi yang di-mask, dan `-100` (`ignore_index`) pada posisi lainnya.

2. **Forward Pass (** `model/model.py` **):**

```
out = model(input_ids=..., attention_mask=..., token_type_ids=..., labels=...)
```

`out.logits` berdimensi `(B, L, V)` dengan $V = \text{vocab_size}$ (`30000`).

Di dalam model, `out.loss` dihitung menggunakan

```
F.cross_entropy(logits.view(-1, V), labels.view(-1), ignore_index=-100)
```

yang meratakan tensor menjadi `(B * L, V)` terhadap target `(B * L,)` dan menghasilkan skalar `()` .

3. **Perhitungan Akurasi MLM & Akumulasi Eksak (** `training/metrics.py` **):**

Prediksi diambil dari indeks probabilitas tertinggi pada dimensi kosakata:

$$\hat{Y} = \arg\max_{v \in V}(\text{logits}) \in \mathbb{Z}^{B \times L}$$

Mask boolean posisi evaluasi:

$$M = (\text{labels} \neq -100) \in \{0, 1\}^{B \times L}$$

Jumlah token ter-mask pada batch b: $N_b = \sum M$.

Jumlah tebakan benar pada batch b: $C_b = \sum(\hat{Y}[M] == \text{labels}[M])$.

Mengapa dibobot per token (`EpochMetricTracker` **)?** Karena pemilihan mask 15% bersifat stokastik (Bernoulli), setiap batch memiliki N_b yang sedikit berbeda. Rata-rata loss dan akurasi satu epoch dihitung secara eksak dengan menjumlahkan seluruh loss yang dikali N_b dibagi $\sum N_b$:

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

$$\mathcal{L}_{\text{epoch}} = \frac{\sum_b \mathcal{L}_b \cdot N_b}{\sum_b N_b}, \quad \text{Acc}_{\text{epoch}} = \frac{\sum_b C_b}{\sum_b N_b}, \quad \text{PPL}_{\text{epoch}} = \exp\left(\min(\mathcal{L}_{\text{e}}\right.$$

1. **Pengaman Edge-Case $N_b = 0$ (`NaN` Guard):**

   Pada kalimat yang sangat pendek di WikiText-2, ada peluang kecil sebuah batch memiliki $N_b = 0$ (semua `labels == -100`). Pada kondisi ini, `F.cross_entropy(..., ignore_index=-100)` dengan reduksi `mean` akan membagi `0 / 0` dan menghasilkan `NaN` . Jika di-backward, seluruh bobot model akan rusak menjadi `NaN` .

   Di `train_one_epoch` , kita memeriksa `if num_masked == 0 or torch.isnan(loss): continue` sebelum memanggil `.backward()` .

B. Custom Linear Warmup + Linear Decay Scheduler (`training/utils.py`)

Menggunakan `torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)` di mana pengali learning rate $\lambda(t)$ pada step ke-t ($0 \le t \le T_{\text{total}}$) didefinisikan sebagai:

$$\lambda(t) = \begin{cases} \frac{t}{\max(1, T_{\text{warmup}})} & \text{jika } t < T_{\text{warmup}} \\ \max\left(0.0, \frac{T_{\text{total}} - t}{\max(1, T_{\text{total}} - T_{\text{warmup}})}\right) & \text{jika } t \ge T_{\text{warmup}} \end{cases}$$

Sehingga learning rate aktual pada step t adalah $\eta_t = \eta_{\text{awal}} \cdot \lambda(t)$.

2. Implementasi Kode Modul `training/`

Berikut adalah kode lengkap untuk `training/__init__.py` , `training/utils.py` , `training/metrics.py` , dan `training/train.py` .

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Pastikan file ini mengekspor fungsi utama agar mudah di-import oleh modul pengujian maupun skrip luar:

Python

```python
"""Package training untuk BERT-from-Scratch (Kelompok 12)."""

from training.metrics import EpochMetricTracker, compute_mlm_accuracy, compute_perp
from training.utils import (
    TrainingLogger,
    build_optimizer,
    build_scheduler,
    get_device,
    load_checkpoint,
    save_checkpoint,
    set_seed,
)

__all__ = [
    "set_seed",
    "get_device",
    "build_optimizer",
    "build_scheduler",
    "save_checkpoint",
    "load_checkpoint",
    "TrainingLogger",
    "compute_mlm_accuracy",
    "compute_perplexity",
    "EpochMetricTracker",
]
```

B. `training/utils.py`

Simpan percakapan ini

Python

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
Utilitas pelatihan BERT-from-Scratch (Kelompok 12):
- Reproducibility (set_seed) & deteksi hardware (get_device)
- Konstruksi AdamW dengan pemisahan weight decay (build_optimizer)
- Custom Linear Warmup + Linear Decay Scheduler dari nol (build_scheduler)
- Manajemen Checkpoint (save_checkpoint, load_checkpoint)
- Pencatatan riwayat metrik ke JSON & CSV (TrainingLogger)
"""

from __future__ import annotations

import csv
import json
import random
from pathlib import Path
from typing import Any

import numpy as np
import torch
import torch.nn as nn
from torch.optim import AdamW, Optimizer
from torch.optim.lr_scheduler import LambdaLR


def set_seed(seed: int = 42) -> None:
    """Mengunci seluruh generator angka acak agar eksperimen dapat direproduksi."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False


def get_device() -> torch.device:
    """Mengembalikan device 'cuda' jika GPU tersedia, selain itu 'cpu'."""
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


def build_optimizer(model: nn.Module, cfg: dict[str, Any]) -> Optimizer:
    """
    Membangun optimizer AdamW sesuai spesifikasi BERT:
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    train_cfg = cfg["training"]
    lr = float(train_cfg.get("learning_rate", 5e-4))
    weight_decay = float(train_cfg.get("weight_decay", 0.01))
    opt_name = str(train_cfg.get("optimizer", "adamw")).lower()

    if opt_name != "adamw":
        raise ValueError(f"Optimizer '{opt_name}' tidak didukung. Gunakan 'adamw' s

    no_decay_keywords = ("bias", "LayerNorm.weight", "layer_norm.weight", "norm.wei

    decay_params = []
    no_decay_params = []

    for name, param in model.named_parameters():
        if not param.requires_grad:
            continue
        if any(nd in name for nd in no_decay_keywords) or param.ndim < 2:
            no_decay_params.append(param)
        else:
            decay_params.append(param)

    optimizer_grouped_parameters = [
        {"params": decay_params, "weight_decay": weight_decay},
        {"params": no_decay_params, "weight_decay": 0.0},
    ]

    optimizer = AdamW(
        optimizer_grouped_parameters,
        lr=lr,
        betas=(0.9, 0.999),
        eps=1e-8,
    )
    return optimizer


def build_scheduler(
    optimizer: Optimizer,
    total_steps: int,
    warmup_steps: int,
) -> LambdaLR:
    """
    Membangun learning rate scheduler Linear Warmup + Linear Decay dari nol
    menggunakan torch.optim.lr_scheduler.LambdaLR.
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    """
    total_steps = max(1, int(total_steps))
    warmup_steps = max(0, int(warmup_steps))

    def lr_lambda(current_step: int) -> float:
        if warmup_steps > 0 and current_step < warmup_steps:
            return float(current_step) / float(max(1, warmup_steps))
        progress = float(total_steps - current_step) / float(
            max(1, total_steps - warmup_steps)
        )
        return max(0.0, progress)

    return LambdaLR(optimizer, lr_lambda=lr_lambda)


def save_checkpoint(
    filepath: str | Path,
    epoch: int,
    model: nn.Module,
    optimizer: Optimizer | None = None,
    scheduler: LambdaLR | None = None,
    scaler: torch.amp.GradScaler | None = None,
    best_val_loss: float = float("inf"),
    cfg: dict[str, Any] | None = None,
    history: list[dict[str, Any]] | None = None,
) -> Path:
    """Menyimpan state pelatihan secara lengkap ke file .pt."""
    filepath = Path(filepath)
    filepath.parent.mkdir(parents=True, exist_ok=True)

    state = {
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "best_val_loss": best_val_loss,
        "config": cfg,
        "history": history or [],
    }
    if optimizer is not None:
        state["optimizer_state_dict"] = optimizer.state_dict()
    if scheduler is not None:
        state["scheduler_state_dict"] = scheduler.state_dict()
    if scaler is not None:
        state["scaler_state_dict"] = scaler.state_dict()
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```python
def load_checkpoint(
    filepath: str | Path,
    model: nn.Module,
    optimizer: Optimizer | None = None,
    scheduler: LambdaLR | None = None,
    scaler: torch.amp.GradScaler | None = None,
    device: torch.device | str = "cpu",
) -> dict[str, Any]:
    """Memuat state pelatihan dari file checkpoint .pt ke dalam objek model/optimi
    filepath = Path(filepath)
    if not filepath.exists():
        raise FileNotFoundError(f"File checkpoint tidak ditemukan: {filepath}")

    checkpoint = torch.load(filepath, map_location=device, weights_only=False)
    model.load_state_dict(checkpoint["model_state_dict"])

    if optimizer is not None and "optimizer_state_dict" in checkpoint:
        optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
    if scheduler is not None and "scheduler_state_dict" in checkpoint:
        scheduler.load_state_dict(checkpoint["scheduler_state_dict"])
    if scaler is not None and "scaler_state_dict" in checkpoint:
        scaler.load_state_dict(checkpoint["scaler_state_dict"])

    return checkpoint


class TrainingLogger:
    """
    Mencatat dan mengekspor riwayat metrik setiap epoch ke format JSON dan CSV
    di direktori cfg['paths']['logs_dir'].
    """

    def __init__(self, logs_dir: str | Path, run_name: str = "train_log") -> None:
        self.logs_dir = Path(logs_dir)
        self.logs_dir.mkdir(parents=True, exist_ok=True)
        self.run_name = run_name
        self.json_path = self.logs_dir / f"{run_name}.json"
        self.csv_path = self.logs_dir / f"{run_name}.csv"
        self.history: list[dict[str, Any]] = []
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```python
    def save(self, extra_metadata: dict[str, Any] | None = None) -> None:
        """Menyimpan seluruh history ke file JSON dan CSV."""
        payload: dict[str, Any] = {
            "run_name": self.run_name,
            "num_epochs_logged": len(self.history),
            "history": self.history,
        }
        if extra_metadata:
            payload.update(extra_metadata)

        with open(self.json_path, "w", encoding="utf-8") as f:
            json.dump(payload, f, indent=2)

        if self.history:
            fieldnames = list(self.history[0].keys())
            with open(self.csv_path, "w", newline="", encoding="utf-8") as f:
                writer = csv.DictWriter(f, fieldnames=fieldnames)
                writer.writeheader()
                writer.writerows(self.history)
```

C. training/metrics.py

Python

```python
"""
training/metrics.py
Perhitungan metrik evaluasi Masked Language Modeling (MLM) dari nol:
- Top-1 Masked Token Accuracy (hanya pada posisi labels != ignore_index)
- Perplexity (exp(loss)) dengan pengaman overflow
- EpochMetricTracker untuk akumulasi metrik eksak per token sepanjang epoch
"""

from __future__ import annotations

import math
import torch
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    labels: torch.Tensor,
    ignore_index: int = -100,
) -> tuple[int, int, float]:
    """
    Menghitung akurasi prediksi top-1 pada posisi token yang di-mask.

    Args:
        logits: Tensor float berukuran (B, L, V)
        labels: Tensor long berukuran (B, L) dengan nilai ignore_index (-100) pada
        ignore_index: Nilai label yang diabaikan (default: -100 sesuai SPEC.md)

    Returns:
        Tuple (num_correct, num_masked_tokens, accuracy_float)
    """
    with torch.no_grad():
        mask = labels != ignore_index  # (B, L) boolean
        num_masked = int(mask.sum().item())

        if num_masked == 0:
            return 0, 0, 0.0

        predictions = logits.argmax(dim=-1)  # (B, L)
        num_correct = int((predictions[mask] == labels[mask]).sum().item())
        accuracy = float(num_correct) / float(num_masked)
        return num_correct, num_masked, accuracy


def compute_perplexity(avg_loss: float, max_loss_cap: float = 20.0) -> float:
    """
    Menghitung Perplexity = exp(avg_loss).
    Dilengkapi batas atas (max_loss_cap=20.0) untuk mencegah OverflowError
    ketika loss masih tinggi di awal pelatihan.
    """
    if math.isnan(avg_loss) or math.isinf(avg_loss):
        return float("inf")
    safe_loss = min(float(avg_loss), max_loss_cap)
    return math.exp(safe_loss)


class EpochMetricTracker:
    """
    Pelacak metrik selama 1 epoch penuh.
    Mengakumulasi loss secara tertimbang terhadap jumlah token ter-mask (num_masked
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```python
    def __init__(self, ignore_index: int = -100) -> None:
        self.ignore_index = ignore_index
        self.reset()

    def reset(self) -> None:
        self.total_weighted_loss: float = 0.0
        self.total_correct: int = 0
        self.total_masked_tokens: int = 0
        self.valid_batches: int = 0
        self.skipped_batches: int = 0

    def update(
        self,
        loss_val: float,
        logits: torch.Tensor,
        labels: torch.Tensor,
    ) -> tuple[int, int, float]:
        """
        Memperbarui statistik dengan hasil satu batch.
        Mengembalikan (num_correct, num_masked, batch_acc).
        """
        num_correct, num_masked, batch_acc = compute_mlm_accuracy(
            logits, labels, ignore_index=self.ignore_index
        )

        if num_masked == 0 or math.isnan(loss_val) or math.isinf(loss_val):
            self.skipped_batches += 1
            return 0, 0, 0.0

        self.total_weighted_loss += float(loss_val) * num_masked
        self.total_correct += num_correct
        self.total_masked_tokens += num_masked
        self.valid_batches += 1
        return num_correct, num_masked, batch_acc

    def compute(self) -> dict[str, float | int]:
        """Menghasilkan ringkasan metrik di akhir epoch."""
        if self.total_masked_tokens == 0:
            avg_loss = 0.0
            accuracy = 0.0
            perplexity = float("inf")
        else:
            avg_loss = self.total_weighted_loss / float(self.total_masked_tokens)
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        return {
            "loss": round(avg_loss, 6),
            "accuracy": round(accuracy, 6),
            "perplexity": round(perplexity, 4),
            "masked_tokens": self.total_masked_tokens,
            "valid_batches": self.valid_batches,
            "skipped_batches": self.skipped_batches,
        }
```

D. training/train.py

Python

```
"""
training/train.py
Entry-point pelatihan, evaluasi, visualisasi kurva, dan CLI Hyperparameter Tuning
untuk proyek BERT-from-Scratch (Kelompok 12).
"""

from __future__ import annotations

import argparse
import copy
import sys
import time
from pathlib import Path
from typing import Any

import matplotlib
matplotlib.use("Agg")  # Aman dijalankan tanpa GUI (Colab / Server / Terminal)
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.optim import Optimizer
from torch.optim.lr_scheduler import LambdaLR
from torch.utils.data import DataLoader
from tqdm.auto import tqdm
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
    sys.path.insert(0, str(PROJECT_ROOT))

from data.data import MLMDataset, load_dataset, make_dataloader
from data.tokenizer import WordTokenizer, build_tokenizer, load_config
from model.model import BERT
from training.metrics import EpochMetricTracker
from training.utils import (
    TrainingLogger,
    build_optimizer,
    build_scheduler,
    get_device,
    load_checkpoint,
    save_checkpoint,
    set_seed,
)


def train_one_epoch(
    model: nn.Module,
    dataloader: DataLoader,
    optimizer: Optimizer,
    scheduler: LambdaLR,
    scaler: torch.amp.GradScaler,
    device: torch.device,
    cfg: dict[str, Any],
    epoch: int,
    max_steps: int | None = None,
) -> dict[str, Any]:
    """
    Menjalankan 1 epoch pelatihan penuh.
    Dilengkapi:
    - Mixed Precision (FP16 AMP) otomatis jika device == 'cuda' dan mixed_precisior
    - Pengaman NaN loss jika kebetulan suatu batch tidak memiliki token ter-mask
    - Gradient clipping sebelum optimizer.step()
    """
    model.train()
    train_cfg = cfg["training"]
    ignore_index = int(train_cfg.get("ignore_index", -100))
    grad_clip = float(train_cfg.get("gradient_clip", 1.0))
    use_amp = bool(train_cfg.get("mixed_precision", True)) and (device.type == "cuc

    tracker = EpochMetricTracker(ignore_index=ignore_index)
    total_batches = len(dataloader) if max_steps is None else min(len(dataloader),
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        total=total_batches,
        desc=f"Epoch {epoch:02d} [Train]",
        leave=False,
    )

    for step, batch in pbar:
        if max_steps is not None and step > max_steps:
            break

        input_ids = batch["input_ids"].to(device, non_blocking=True)
        attention_mask = batch["attention_mask"].to(device, non_blocking=True)
        token_type_ids = batch["token_type_ids"].to(device, non_blocking=True)
        labels = batch["labels"].to(device, non_blocking=True)

        # Guard 1: Cek apakah ada minimal 1 token yang di-mask pada batch ini
        has_masked_tokens = bool((labels != ignore_index).any().item())
        if not has_masked_tokens:
            tracker.skipped_batches += 1
            continue

        optimizer.zero_grad(set_to_none=True)

        with torch.amp.autocast(device_type=device.type, enabled=use_amp):
            out = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                token_type_ids=token_type_ids,
                labels=labels,
            )
            logits = out.logits
            loss = out.loss

        # Guard 2: Lewati backward pass jika loss bernilai NaN atau Inf
        if loss is None or torch.isnan(loss) or torch.isinf(loss):
            tracker.skipped_batches += 1
            continue

        # Backward pass dengan GradScaler (otomatis no-op jika enabled=False)
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)

        if grad_clip > 0.0:
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=grad_clip)
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
            scheduler.step()

            loss_val = float(loss.detach().item())
            _, _, batch_acc = tracker.update(loss_val, logits.detach(), labels)
            current_lr = float(optimizer.param_groups[0]["lr"])

            pbar.set_postfix(
                loss=f"{loss_val:.4f}",
                acc=f"{batch_acc * 100:.2f}%",
                lr=f"{current_lr:.2e}",
            )

        summary = tracker.compute()
        summary["lr"] = float(optimizer.param_groups[0]["lr"])
        return summary


@torch.no_grad()
def evaluate(
    model: nn.Module,
    dataloader: DataLoader,
    device: torch.device,
    cfg: dict[str, Any],
    split_name: str = "Val",
    max_steps: int | None = None,
) -> dict[str, Any]:
    """
    Mengevaluasi model pada split validation atau test tanpa menghitung gradien.
    """
    model.eval()
    train_cfg = cfg["training"]
    ignore_index = int(train_cfg.get("ignore_index", -100))
    use_amp = bool(train_cfg.get("mixed_precision", True)) and (device.type == "cu

    tracker = EpochMetricTracker(ignore_index=ignore_index)
    total_batches = len(dataloader) if max_steps is None else min(len(dataloader),

    pbar = tqdm(
        enumerate(dataloader, start=1),
        total=total_batches,
        desc=f"Eval [{split_name}]",
        leave=False,
    )
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
            break

        input_ids = batch["input_ids"].to(device, non_blocking=True)
        attention_mask = batch["attention_mask"].to(device, non_blocking=True)
        token_type_ids = batch["token_type_ids"].to(device, non_blocking=True)
        labels = batch["labels"].to(device, non_blocking=True)

        if not bool((labels != ignore_index).any().item()):
            tracker.skipped_batches += 1
            continue

        with torch.amp.autocast(device_type=device.type, enabled=use_amp):
            out = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                token_type_ids=token_type_ids,
                labels=labels,
            )
            logits = out.logits
            loss = out.loss

        if loss is None or torch.isnan(loss) or torch.isinf(loss):
            tracker.skipped_batches += 1
            continue

        loss_val = float(loss.item())
        _, _, batch_acc = tracker.update(loss_val, logits, labels)
        pbar.set_postfix(loss=f"{loss_val:.4f}", acc=f"{batch_acc * 100:.2f}%")

    return tracker.compute()


def plot_training_curves(
    history: list[dict[str, Any]],
    figures_dir: str | Path,
    filename: str = "training_curves.png",
) -> Path | None:
    """
    Membuat dan menyimpan grafik 3 panel:
    1. Train vs Val Loss
    2. Train vs Val MLM Top-1 Accuracy (%)
    3. Train vs Val Perplexity
    """
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
figures_dir = Path(figures_dir)
figures_dir.mkdir(parents=True, exist_ok=True)
out_path = figures_dir / filename

epochs = [row["epoch"] for row in history]
train_loss = [row["train_loss"] for row in history]
val_loss = [row["val_loss"] for row in history]
train_acc = [row["train_accuracy"] * 100.0 for row in history]
val_acc = [row["val_accuracy"] * 100.0 for row in history]
train_ppl = [row["train_perplexity"] for row in history]
val_ppl = [row["val_perplexity"] for row in history]

fig, axes = plt.subplots(1, 3, figsize=(16, 4.8))

# Panel 1: MLM Loss
axes[0].plot(epochs, train_loss, marker="o", linewidth=2, label="Train Loss")
axes[0].plot(epochs, val_loss, marker="s", linewidth=2, label="Val Loss")
axes[0].set_title("MLM Cross-Entropy Loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Loss")
axes[0].grid(True, linestyle="--", alpha=0.5)
axes[0].legend()

# Panel 2: MLM Top-1 Accuracy
axes[1].plot(epochs, train_acc, marker="o", linewidth=2, label="Train Accuracy'
axes[1].plot(epochs, val_acc, marker="s", linewidth=2, label="Val Accuracy")
axes[1].set_title("MLM Masked Token Accuracy (%)")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy (%)")
axes[1].grid(True, linestyle="--", alpha=0.5)
axes[1].legend()

# Panel 3: Perplexity
axes[2].plot(epochs, train_ppl, marker="o", linewidth=2, label="Train PPL")
axes[2].plot(epochs, val_ppl, marker="s", linewidth=2, label="Val PPL")
axes[2].set_title("MLM Perplexity (exp(Loss))")
axes[2].set_xlabel("Epoch")
axes[2].set_ylabel("Perplexity")
axes[2].grid(True, linestyle="--", alpha=0.5)
axes[2].legend()

fig.suptitle("BERT-from-Scratch (Kaggle, 12) - Training Progress", fontsize=13
fig.tight_layout()
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```python
def parse_args() -> argparse.Namespace:
    """Mendefinisikan argumen CLI untuk pelatihan standar maupun Hyperparameter Tur
    parser = argparse.ArgumentParser(
        description="Pelatihan & Hyperparameter Tuning BERT-from-Scratch (Kelompok
    )
    parser.add_argument("--config", type=str, default="config.yaml", help="Path ke
    parser.add_argument("--run-name", type=str, default="bert_mlm_baseline", help='

    # Override Hyperparameter (Opsional untuk eksperimen tuning)
    parser.add_argument("--lr", type=float, default=None, help="Override learning_r
    parser.add_argument("--batch-size", type=int, default=None, help="Override batc
    parser.add_argument("--epochs", type=int, default=None, help="Override num_epoc
    parser.add_argument("--weight-decay", type=float, default=None, help="Override
    parser.add_argument("--warmup-ratio", type=float, default=None, help="Override
    parser.add_argument("--grad-clip", type=float, default=None, help="Override gra
    parser.add_argument("--dropout", type=float, default=None, help="Override model
    parser.add_argument("--seed", type=int, default=None, help="Override random see
    parser.add_argument("--no-amp", action="store_true", help="Matikan FP16 Mixed P

    # Kontrol Eksekusi & Debugging
    parser.add_argument(
        "--max-steps",
        type=int,
        default=None,
        help="Batasi jumlah batch per epoch (berguna untuk smoke test cepat di CPU/
    )
    parser.add_argument("--resume", type=str, default=None, help="Path checkpoint .
    parser.add_argument("--eval-test", action="store_true", help="Jalankan evaluasi

    return parser.parse_args()


def apply_cli_overrides(cfg: dict[str, Any], args: argparse.Namespace) -> dict[str,
    """Menyalin konfigurasi dan menerapkan override dari CLI tanpa mengubah file co
    cfg = copy.deepcopy(cfg)
    if args.lr is not None:
        cfg["training"]["learning_rate"] = float(args.lr)
    if args.batch_size is not None:
        cfg["training"]["batch_size"] = int(args.batch_size)
    if args.epochs is not None:
        cfg["training"]["num_epochs"] = int(args.epochs)
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        cfg["training"]["warmup_ratio"] = float(args.warmup_ratio)
    if args.grad_clip is not None:
        cfg["training"]["gradient_clip"] = float(args.grad_clip)
    if args.dropout is not None:
        cfg["model"]["dropout"] = float(args.dropout)
    if args.seed is not None:
        cfg["training"]["seed"] = int(args.seed)
        cfg["project"]["seed"] = int(args.seed)
    if args.no_amp:
        cfg["training"]["mixed_precision"] = False
    return cfg


def prepare_tokenizer_and_dataloaders(
    cfg: dict[str, Any],
) -> tuple[WordTokenizer, DataLoader, DataLoader, DataLoader | None]:
    """Memuat dataset WikiText-2, membangun/memuat WordTokenizer, dan membuat Datal
    print("[1/4] Memuat dataset WikiText-2...")
    lines = load_dataset(cfg)

    vocab_path = Path(cfg["paths"]["vocab_path"])
    if vocab_path.exists():
        print(f"[2/4] Memuat tokenizer dari '{vocab_path}'...")
        tokenizer = WordTokenizer.load(cfg)
    else:
        print(f"[2/4] Membangun tokenizer baru dan menyimpan ke '{vocab_path}'...")
        tokenizer = build_tokenizer(cfg, texts=lines["train"])

    print("[3/4] Menyiapkan MLMDataset & DataLoader...")
    train_ds = MLMDataset(lines["train"], tokenizer, cfg, split="train")
    val_ds = MLMDataset(lines["validation"], tokenizer, cfg, split="validation")
    test_ds = MLMDataset(lines["test"], tokenizer, cfg, split="test") if "test" in

    train_loader = make_dataloader(train_ds, cfg, shuffle=True)
    val_loader = make_dataloader(val_ds, cfg, shuffle=False)
    test_loader = make_dataloader(test_ds, cfg, shuffle=False) if test_ds is not No

    return tokenizer, train_loader, val_loader, test_loader


def main() -> None:
    args = parse_args()
    raw_cfg = load_config(args.config)
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
set_seed(seed)
device = get_device()
use_amp = bool(cfg["training"].get("mixed_precision", True)) and (device.type =

print("=" * 72)
print(f"BERT-FROM-SCRATCH (KELOMPOK 12) - RUN: {args.run_name}")
print("=" * 72)
print(
    f"Device          : {device} (AMP FP16: {use_amp})\n"
    f"Seed            : {seed}\n"
    f"Hyperparameters : bs={cfg['training']['batch_size']}, "
    f"lr={cfg['training']['learning_rate']}, "
    f"wd={cfg['training']['weight_decay']}, "
    f"epochs={cfg['training']['num_epochs']}, "
    f"warmup={cfg['training']['warmup_ratio']}, "
    f"clip={cfg['training']['gradient_clip']}"
)
print("-" * 72)

_, train_loader, val_loader, test_loader = prepare_tokenizer_and_dataloaders(cf

print("[4/4] Menginisialisasi arsitektur BERT...")
model = BERT.from_config(cfg).to(device)
total_params = model.count_parameters(trainable_only=False)
trainable_params = model.count_parameters(trainable_only=True)
print(f"Total Parameter : {total_params:,} (Trainable: {trainable_params:,})")

num_epochs = int(cfg["training"]["num_epochs"])
steps_per_epoch = (
    len(train_loader)
    if args.max_steps is None
    else min(len(train_loader), args.max_steps)
)
total_steps = max(1, num_epochs * steps_per_epoch)
warmup_ratio = float(cfg["training"].get("warmup_ratio", 0.10))
warmup_steps = int(total_steps * warmup_ratio)

optimizer = build_optimizer(model, cfg)
scheduler = build_scheduler(optimizer, total_steps=total_steps, warmup_steps=wa
scaler = torch.amp.GradScaler(device.type, enabled=use_amp)

ckpt_dir = Path(cfg["paths"]["checkpoint_dir"])
logs_dir = Path(cfg["paths"]["logs_dir"])
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        "best_model.pt" if args.run_name == "bert_mlm_baseline" else f"best_model_
    )
    last_ckpt_path = ckpt_dir / (
        "last_model.pt" if args.run_name == "bert_mlm_baseline" else f"last_model_
    )

    logger = TrainingLogger(logs_dir=logs_dir, run_name=args.run_name)
    start_epoch = 1
    best_val_loss = float("inf")

    if args.resume:
        print(f"Melanjutkan pelatihan dari checkpoint: {args.resume}")
        ckpt = load_checkpoint(
            args.resume,
            model=model,
            optimizer=optimizer,
            scheduler=scheduler,
            scaler=scaler,
            device=device,
        )
        start_epoch = int(ckpt.get("epoch", 0)) + 1
        best_val_loss = float(ckpt.get("best_val_loss", float("inf")))
        logger.history = list(ckpt.get("history", []))

    print(
        f"Total Steps     : {total_steps} ({steps_per_epoch} steps/epoch x {num_epo
        f"warmup={warmup_steps} steps)"
    )
    print("=" * 72)

    for epoch in range(start_epoch, num_epochs + 1):
        t0 = time.time()

        train_metrics = train_one_epoch(
            model=model,
            dataloader=train_loader,
            optimizer=optimizer,
            scheduler=scheduler,
            scaler=scaler,
            device=device,
            cfg=cfg,
            epoch=epoch,
            max_steps=args.max_steps,
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
        model=model,
        dataloader=val_loader,
        device=device,
        cfg=cfg,
        split_name="Val",
        max_steps=args.max_steps,
    )

    elapsed = round(time.time() - t0, 2)
    is_best = val_metrics["loss"] < best_val_loss
    if is_best:
        best_val_loss = float(val_metrics["loss"])

    record = {
        "epoch": epoch,
        "train_loss": train_metrics["loss"],
        "train_accuracy": train_metrics["accuracy"],
        "train_perplexity": train_metrics["perplexity"],
        "val_loss": val_metrics["loss"],
        "val_accuracy": val_metrics["accuracy"],
        "val_perplexity": val_metrics["perplexity"],
        "lr": train_metrics["lr"],
        "epoch_time_sec": elapsed,
        "is_best": is_best,
    }
    logger.log_epoch(record)

    # Simpan checkpoint terakhir & terbaik
    save_checkpoint(
        filepath=last_ckpt_path,
        epoch=epoch,
        model=model,
        optimizer=optimizer,
        scheduler=scheduler,
        scaler=scaler,
        best_val_loss=best_val_loss,
        cfg=cfg,
        history=logger.history,
    )
    if is_best:
        save_checkpoint(
            filepath=best_ckpt_path,
            epoch=epoch,
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
                scaler=scaler,
                best_val_loss=best_val_loss,
                cfg=cfg,
                history=logger.history,
            )

        best_marker = " *BEST*" if is_best else ""
        print(
            f"Epoch {epoch:02d}/{num_epochs:02d} ({elapsed:.1f}s) | "
            f"Train Loss: {train_metrics['loss']:.4f} | "
            f"Train Acc: {train_metrics['accuracy'] * 100:.2f}% | "
            f"Val Loss: {val_metrics['loss']:.4f} | "
            f"Val Acc: {val_metrics['accuracy'] * 100:.2f}% | "
            f"Val PPL: {val_metrics['perplexity']:.2f}{best_marker}"
        )

    # Plot grafik kurva pelatihan
    curve_filename = (
        "training_curves.png"
        if args.run_name == "bert_mlm_baseline"
        else f"training_curves_{args.run_name}.png"
    )
    plot_path = plot_training_curves(logger.history, figures_dir=figures_dir, filer

    # Evaluasi akhir menggunakan bobot terbaik (best_model.pt)
    extra_meta: dict[str, Any] = {
        "best_val_loss": best_val_loss,
        "best_checkpoint": str(best_ckpt_path),
        "figures_path": str(plot_path) if plot_path else None,
        "hyperparameters": cfg["training"],
    }

    if best_ckpt_path.exists():
        load_checkpoint(best_ckpt_path, model=model, device=device)
        final_val = evaluate(
            model=model,
            dataloader=val_loader,
            device=device,
            cfg=cfg,
            split_name="Best-Val",
            max_steps=args.max_steps,
        )
        extra_meta["final_best_val_metrics"] = final_val
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

```
            f"Loss: {final_val['loss']:.4f} | "
            f"Acc: {final_val['accuracy'] * 100:.2f}% | "
            f"PPL: {final_val['perplexity']:.2f}"
        )

        if args.eval_test and test_loader is not None:
            final_test = evaluate(
                model=model,
                dataloader=test_loader,
                device=device,
                cfg=cfg,
                split_name="Test",
                max_steps=args.max_steps,
            )
            extra_meta["final_test_metrics"] = final_test
            print(
                f"[Best Checkpoint Eval - Test] "
                f"Loss: {final_test['loss']:.4f} | "
                f"Acc: {final_test['accuracy'] * 100:.2f}% | "
                f"PPL: {final_test['perplexity']:.2f}"
            )

    logger.save(extra_metadata=extra_meta)
    print("=" * 72)
    print(f"Log JSON tersimpan di : {logger.json_path}")
    print(f"Log CSV tersimpan di  : {logger.csv_path}")
    if plot_path:
        print(f"Kurva tersimpan di    : {plot_path}")
    print(f"Model terbaik di      : {best_ckpt_path}")
    print("=" * 72)


if __name__ == "__main__":
    main()
```

3. Panduan Menjalankan & Menguji Modul (`Smoke Test` & `Hyperparameter Tuning`)

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Sebelum menjalankan training 10 epoch penuh di GPU, verifikasi bahwa seluruh pipa (`data` → `model` → `training` → `checkpoint` → `plot`) bebas bug dengan membatasi `--epochs 2 --max-steps 2` :

Bash

```
python -m training.train --config config.yaml --epochs 2 --max-steps 2 --run-name s
```

Perintah di atas akan:

1. Menjalankan 2 batch training dan 2 batch validasi selama 2 epoch.
2. Menyimpan checkpoint `checkpoints/best_model_smoke_test.pt` dan `last_model_smoke_test.pt` .
3. Menyimpan log `outputs/logs/smoke_test.json` & `.csv` .
4. Membuat plot `outputs/figures/training_curves_smoke_test.png` .

B. Pelatihan Baseline Penuh (Sesuai Default `SPEC.md` di Google Colab T4)

Bash

```
python -m training.train --config config.yaml --run-name bert_mlm_baseline --eval-t
```

(Menghasilkan `checkpoints/best_model.pt` , `outputs/logs/bert_mlm_baseline.json` , *dan* `outputs/figures/training_curves.png` *).*

C. Eksperimen Hyperparameter Tuning (Tugas Bima — Anggota 3)

Simpan percakapan ini

Untuk membandingkan variasi hyperparameter di laporan akhir tanpa mengutak-atik

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini

Bash

```
# Eksperimen 1: Baseline (LR 5e-4, Warmup 10%, Weight Decay 0.01)
!python -m training.train --run-name exp1_baseline_lr5e4 --lr 5e-4 --warmup-ratio 0

# Eksperimen 2: Learning Rate Lebih Kecil (LR 3e-4) - Menguji kestabilan konvergens
!python -m training.train --run-name exp2_lr3e4 --lr 3e-4 --warmup-ratio 0.10

# Eksperimen 3: Learning Rate Lebih Agresif (LR 1e-3) + Warmup Lebih Panjang (15%)
!python -m training.train --run-name exp3_lr1e3_warmup15 --lr 1e-3 --warmup-ratio 0

# Eksperimen 4: Regularisasi Lebih Kuat (Weight Decay 0.05, Dropout 0.2)
!python -m training.train --run-name exp4_reg_wd005_drop02 --weight-decay 0.05 --dr
```

Simpan percakapan ini

Gemini mungkin menampilkan info yang tidak akurat, termasuk tentang orang, jadi periksa kembali responsnya.

Kebijakan Privasi Google Persyaratan Layanan Google Privasi Anda & Aplikasi Gemini